

Arquitectura y modelo de datos

Proyecto integrador | Documentación técnica y académica

Meteorología y energía sintética en cinco ciudades de Colombia

Juan Sebastian Clavijo Correa

Institución Universitaria Digital de Antioquia
Infraestructura y Arquitectura para Big Data
30 de septiembre de 2026

Aspecto revisado	Resultado
Periodo y unidad de registro	2025 completo; una ciudad por hora local
Integración	43.800 filas; 32 columnas limpias y 63 enriquecidas
Modelo final	SQLite: 6 tablas y una vista de reconstrucción
Herramientas de las fases previas	DuckDB de EA1, Pandas y GitHub Actions

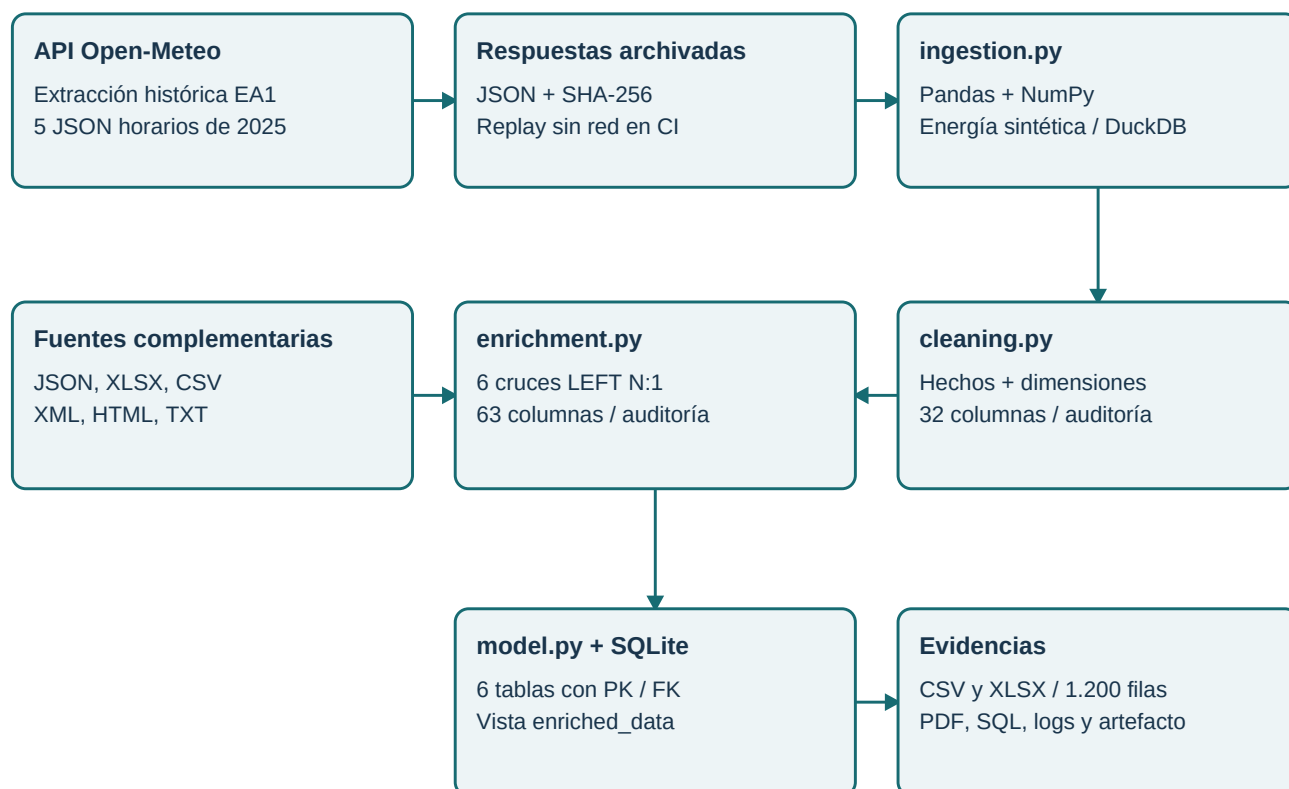
En este trabajo reúno la ingesta, la limpieza y el enriquecimiento que desarrollé durante el curso. Explico cómo se conectan los scripts, cómo quedan organizados los datos y qué comprobaciones hice. La variable de energía fue simulada para el ejercicio; por eso no la uso para sacar conclusiones sobre el consumo eléctrico real ni sobre su relación con el clima.

Repositorio: [JSebastianCCorrea/documentacion_arquitectura_modelo_datos](#)

Lectura: arquitectura (2), ingesta (3), limpieza (4), enriquecimiento (5), modelo ER (6), tablas y campos (7-10), herramientas (11), automatización (12), resultados y recomendaciones (13), bibliografía (14).

1. Visión global de la arquitectura

El proyecto trabaja con datos del clima de Medellín, Bogotá, Cali, Barranquilla y Bucaramanga, registrados por hora durante 2025. A esos datos les agregué una variable de energía simulada. La idea es tener una base organizada para revisar diferencias entre ciudades, fechas y horas. Con este tamaño puedo ejecutar todo en un computador y comprobar cada paso.



GitHub Actions ejecuta la cadena en un runner y conserva los resultados durante 30 días.

Figura 1. Flujo completo. Elaboración propia a partir de los scripts del repositorio. El tramo API-JSON corresponde a la extracción de EA1; la ejecución automatizada actual comienza en las respuestas archivadas.

El proceso tiene cuatro pasos: leer los datos de entrada, limpiarlos, agregar información de otras fuentes y guardar el resultado en SQLite. Cada paso deja un reporte para revisar qué ocurrió. Conservo los archivos originales para poder repetir el proceso y encontrar de dónde sale cada dato.

Esta entrega retoma el trabajo de ingesta de EA1, la limpieza de EA2 y el enriquecimiento guardado en actividad_4. Aquí los reúno en un solo proceso y explico el modelo final de datos.

2. Ingesta y origen de los datos

2.1 Extracción realizada en EA1

En EA1, el script consultó Historical Weather API de Open-Meteo con `requests.get`. Usó las coordenadas de cada ciudad, el periodo del 1 de enero al 31 de diciembre de 2025 y la zona America/Bogota. Solicitó `temperature_2m`, `relative_humidity_2m`, `precipitation` y `wind_speed_10m`, y guardó cada respuesta en JSON. Son datos meteorológicos modelados. Como esa consulta no fijó el parámetro `models`, no puedo asegurar que todos provengan de ERA5 (Open-Meteo, s. f.).

Las coordenadas de la base son las que se enviaron en la consulta. El proveedor puede responder con las de una celda cercana de su malla meteorológica. Los JSON originales permiten revisar esa diferencia y las unidades de cada variable.

2.2 Simulación de energía y carga analítica

Para simular la energía en EA1, usé valores base de 110, 130, 115, 125 y 100 para Medellín, Bogotá, Cali, Barranquilla y Bucaramanga. Cada valor se multiplicó por un factor horario: 1,25 entre las 18 y las 21 h, 1,15 entre las 7 y las 9 h, y 1 en las demás horas. También se aplicaron los factores $1 + |T - 22| \times 0,015$ y $1 + \text{precipitación} \times 0,005$, más un ruido normal con media 1 y desviación 0,05. La semilla 42 de NumPy permite repetir la simulación; el resultado se redondea a dos decimales.

Los datos se guardaron primero en `staging_weather` y luego se organizaron en `dim_city`, `dim_time` y `fact_weather_energy`. En EA1, `CREATE TABLE AS` creó las tablas sin declarar claves primarias o foráneas. Por eso, antes de unirlos, el script de limpieza revisa que las claves no se repitan y que cada referencia tenga su ciudad y su hora.

Tabla de EA1 (DuckDB)	Filas	Función
<code>staging_weather</code>	43.800	Carga inicial: 11 columnas
<code>dim_city</code>	5	Identificador local y coordenadas
<code>dim_time</code>	8.760	Una hora por fila
<code>fact_weather_energy</code>	43.800	Ciudad/hora, meteorología y energía sintética

2.3 Reproducción en esta entrega

En esta entrega, `src/ingestion.py` comprueba la huella SHA-256 de los cinco JSON y vuelve a generar los datos sin conectarse a Internet. Mantiene el orden de ciudades y horas para obtener la misma energía simulada. Comparo cada valor con `src/db/ingestion.db`, que es una copia de `analytics.duckdb` de EA1 y sigue usando DuckDB. El código original está en `docs/antecedentes/ingesta_ea1_original.py`.

3. Preprocesamiento y limpieza

Para la limpieza mantuve las reglas de EA2 en `src/cleaning.py`. El script lee la base sin modificarla, revisa las claves y ordena los registros por ciudad y hora. Asigna `source_row` para identificar cada fila, unifica la escritura de los nombres de ciudad y convierte los campos al tipo que corresponde.

Control	Decisión aplicada
Clave ciudad-hora	Se eliminan duplicados exactos. Si una clave ciudad-hora tiene valores contradictorios, se rechazan todas sus versiones.
Valores ausentes	Se descartan claves inválidas. Los valores que se pueden completar usan la mediana de su ciudad, año y mes. Si falta lluvia o no hay datos válidos para calcular la mediana, se rechaza la fila.
Rangos físicos	Latitud, longitud, humedad y medidas no negativas se validan antes de aceptar un registro.
Atípicos	Regla IQR por ciudad: marcas de revisión, sin recortar ni borrar automáticamente.
Zona horaria	Interpretación local America/Bogota y columna adicional <code>observation_time_utc</code> .
Transformaciones	Z-score de temperatura y energía por ciudad, humedad/100 y <code>log1p</code> de precipitación.

3.1 Resultado de la limpieza

Al revisar la entrada encontré 43.800 filas sin datos nulos ni duplicados. Por eso, en esta ejecución no fue necesario eliminar filas ni completar valores faltantes. Sí quedaron 6.240 filas con alguna marca de dato atípico. Las conservé para poder revisarlas, ya que una marca estadística por sí sola no demuestra que el dato esté mal.

El CSV limpio tiene 32 columnas y coincide byte a byte con la salida de EA2. También se guardan muestras CSV y XLSX de 1.200 filas: hasta 20 por ciudad, año y mes, con semilla 42. El reporte de cobertura indica cuántas filas representa cada grupo y qué peso usar al calcular resultados con la muestra.

3.2 Uso posterior de las variables

Los z-scores y las marcas IQR se calcularon con todos los datos de 2025. Esto sirve para describir la base actual. Si después la uso para predecir, primero debo separar los datos por tiempo y calcular esas transformaciones solo con el conjunto de entrenamiento. En esta actividad no entrené modelos de predicción.

Evidencias: `src/static/auditoria/limpieza/cleaning_report.txt`, `cleaning_metrics.json`, `quality_profile.csv` y `rejected_records.csv`. La copia histórica de auditoría de EA2 se conserva en `ea2_original/` para distinguir el antecedente de la ejecución integrada.

4. Enriquecimiento de seis formatos

Para enriquecer los datos uso el CSV limpio completo. Lo cargo en la tabla `cleaned_data` de `cleaned.duckdb` y lo leo con Pandas. Las seis fuentes están guardadas en el repositorio junto con las respuestas originales y los archivos que registran su origen y sus hashes. Elegí Pandas porque permite leer estos formatos y comprobar los cruces con `merge` (pandas development team, s. f.-a).

Archivo / formato	Clave de cruce	Filas fuente	Aporte
<code>city_geography.json</code>	Ciudad normalizada	5	País, GeoNames y elevación
<code>calendar_2025.xlsx</code>	Fecha local	365	Festivo y fin de semana
<code>solar_daily_2025.csv</code>	Ciudad + fecha	1.825	Radiación y duración solar
<code>municipalities.html</code>	Ciudad normalizada	5	Códigos y nombres DANE
<code>thermal_parameters.xml</code>	Ciudad normalizada	5	Referencia académica 18 °C
<code>hour_bands.txt</code>	Hora local	24	Franja del día

Los seis cruces usan `LEFT JOIN` y `validate="many_to_one"`. Así conservo cada fila de la base y detengo el proceso si una fuente repite la clave que debería ser única. Para comparar nombres de ciudad creo una clave sin tildes, diferencias de mayúsculas ni espacios sobrantes; el nombre original se mantiene. Los cruces diarios usan la fecha de Colombia.

Las 43.800 filas encontraron datos en las seis fuentes y la cantidad de registros se mantuvo. Se agregaron 31 columnas, entre ellas los indicadores `matched_*`, las conversiones de unidades y el estado del enriquecimiento. Para pasar de MJ/m²/día a kWh/m²/día divido entre 3,6; para pasar de segundos a horas, entre 3.600. La referencia de 18 °C se usa como ejemplo de cálculo, sin tratarla como un indicador de confort validado.

4.1 Origen de las fuentes y uso de los datos

Usé cuatro conjuntos de datos externos de tres proveedores: Open-Meteo/GeoNames, Nager.Date y DANE. Los catálogos XML y TXT los preparé para el ejercicio. El HTML se generó a partir del JSON oficial del DANE. En el calendario, 18 festividades se agrupan en 17 fechas. Para Bucaramanga conservé el año DIVIPOLA 2024 que devuelve el servicio, aunque su nombre indique 2025.

Cuando una fila no encuentra información en alguna fuente, `enrichment.py` la conserva con el estado `INCOMPLETE` y deja los campos faltantes sin completar. El paso a SQLite exige `COMPLETE` y se detiene si hay pendientes. Además, los datos solares describen el día completo: no se pueden usar para predecir una hora de ese mismo día antes de que esos datos estén disponibles.

5. Modelo lógico y relaciones

Organicé el modelo en dos tablas de hechos y cuatro dimensiones. Una tabla guarda las mediciones por ciudad y hora; la otra, los datos solares por ciudad y día. Las dimensiones contienen la ciudad, la fecha, la hora y el instante de observación. Separé los datos diarios de los horarios para evitar sumar 24 veces una misma medida solar.

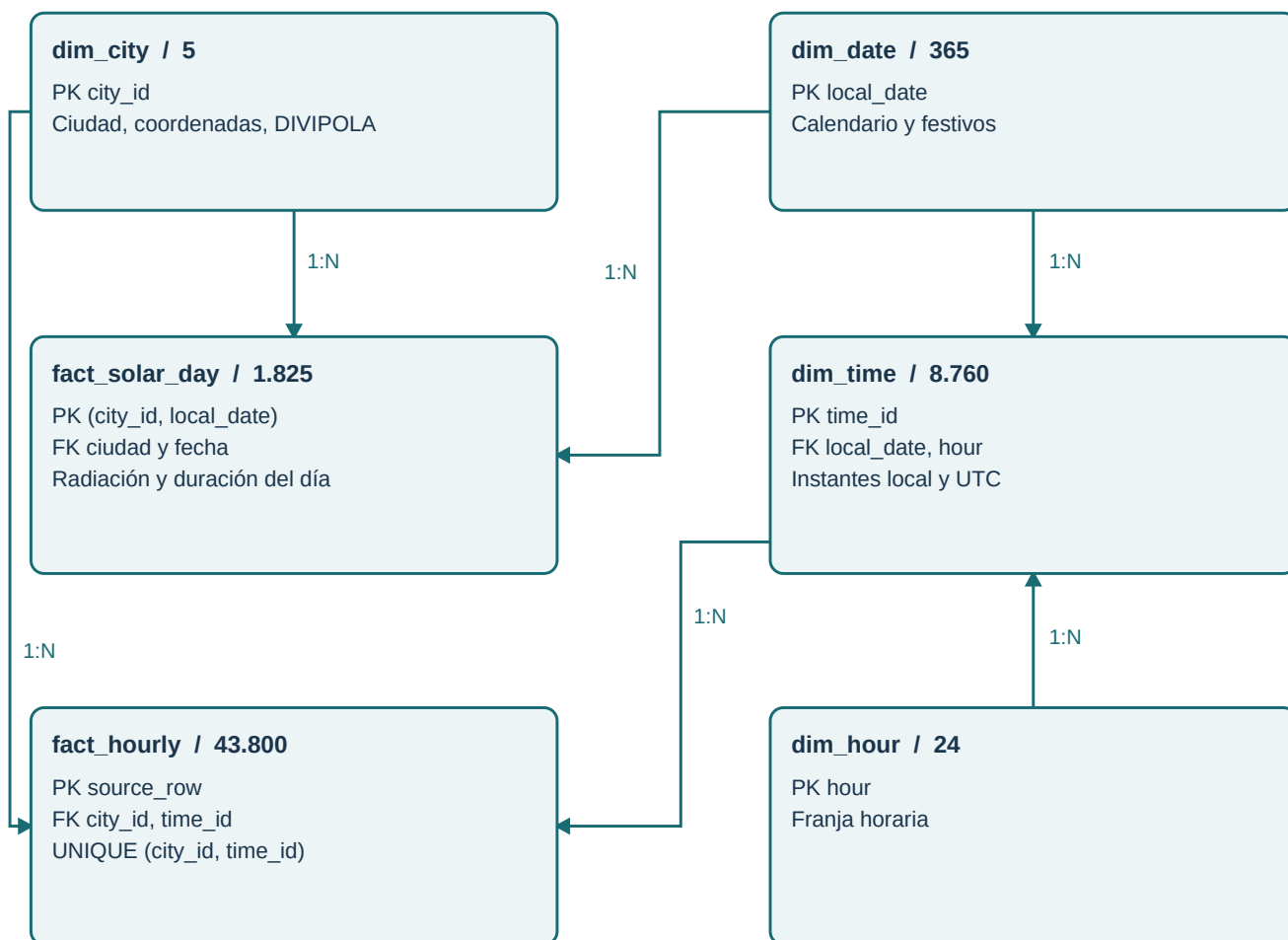


Figura 2. Modelo físico SQLite. Las flechas indican relación de padre a hijo 1:N. Cada hijo tiene una sola fila padre para la clave obligatoria. Elaboración propia; PK y FK corresponden al DDL ejecutado.

fact_hourly se relaciona con dim_city y dim_time. A su vez, dim_time se relaciona con dim_date y dim_hour. fact_solar_day usa dim_city y dim_date. Para consultar todo junto, la vista enriched_data une los datos horarios y solares por ciudad y fecha, sin una clave foránea directa entre las dos tablas de hechos.

model.py activa PRAGMA foreign_keys=ON y crea las tablas STRICT en una base temporal. Luego ejecuta foreign_key_check e integrity_check y compara las 63 columnas de la vista con el CSV. Solo reemplaza analytics.sqlite si todas las comprobaciones pasan. Las reglas NOT NULL, UNIQUE y CHECK también ayudan a detectar datos que no cumplen el esquema (SQLite, s. f.-a).

6. Tablas y campos: ciudad

Los tipos de estas tablas se tomaron de la base creada mediante PRAGMA table_info. Todos los campos son NOT NULL. Los booleanos se guardan como INTEGER y solo aceptan 0 o 1. Los códigos territoriales se guardan como TEXT para conservar ceros iniciales, como en 05001. DICCIONARIO_DATOS.md incluye las unidades y la explicación completa de las 63 variables.

dim_city | 5 filas

Campo	Tipo / clave	Descripción
city_id	INTEGER / PK	Identificador de ciudad en EA1
city	TEXT	Barranquilla, Bogota, Bucaramanga, Cali o Medellin, con la escritura de EA2
latitude	REAL	Latitud solicitada en la extracción original
longitude	REAL	Longitud solicitada en la extracción original
country_code	TEXT	Código de país, CO
geoname_id	INTEGER	Identificador del punto en GeoNames
elevation_m	REAL	Elevación del punto que devuelve la consulta
matched_geography	INTEGER	La ciudad encontró correspondencia en el catálogo geográfico
municipality_code	TEXT	Código DIVIPOLA municipal
municipality_name	TEXT	Nombre municipal informado por DANE
department_code	TEXT	Código departamental
department_name	TEXT	Nombre departamental informado por DANE
divipola_year	INTEGER	Año del registro MPIO_NANO, conservado de la fuente
matched_municipalities	INTEGER	La ciudad encontró correspondencia DIVIPOLA
reference_temperature_c	REAL	Referencia ilustrativa de 18 °C
parameter_scope	TEXT	academic_illustration
matched_thermal	INTEGER	La ciudad encontró referencia térmica

city_id es el identificador que viene de EA1; municipality_code es el código DANE. city y municipality_code son únicos. Los códigos municipales deben tener cinco dígitos y los departamentales, dos. Esta tabla contiene las cinco ciudades del proyecto.

7. Tablas y campos: calendario y tiempo

dim_date | 365 filas

Campo	Tipo / clave	Descripción
local_date	TEXT / PK	Fecha de la observación en Colombia
year	INTEGER	Año local: 2025
month	INTEGER	Mes local: 1 a 12
day	INTEGER	Día local del mes
holiday_name	TEXT	Nombre de festividad o No festivo
is_holiday	INTEGER	La fecha figura como festiva en el calendario
is_weekend	INTEGER	La fecha es sábado o domingo
matched_calendar	INTEGER	La fecha encontró correspondencia en el calendario
is_business_day	INTEGER	Fecha que no es festiva ni fin de semana según calendar

dim_hour | 24 filas

Campo	Tipo / clave	Descripción
hour	INTEGER / PK	Hora local: 0 a 23
day_period	TEXT	madrugada, mañana, tarde o noche
matched_hours	INTEGER	La hora encontró clasificación en el catálogo

dim_time | 8.760 filas

Campo	Tipo / clave	Descripción
time_id	INTEGER / PK	Identificador de la hora en la dimensión temporal de EA1
observation_time	TEXT	Fecha y hora de la observación en America/Bogota
observation_time_utc	TEXT	El mismo instante de observation_time, expresado en UTC
local_date	TEXT / FK	Fecha de la observación en Colombia
hour	INTEGER / FK	Hora local: 0 a 23
timezone	TEXT	America/Bogota

Las fechas se guardan como texto ISO 8601. observation_time usa el desplazamiento -05:00; observation_time_utc guarda el mismo instante en UTC. timezone indica America/Bogota. SQLite no tiene un tipo TIMESTAMPTZ, que sí se usa en la tabla cleaned_data de DuckDB (SQLite, s. f.-b).

8. Tablas y campos: datos solares

fact_solar_day | 1.825 filas

Campo	Tipo / clave	Descripción
city_id	INTEGER / PK / FK	Identificador de ciudad en EA1
local_date	TEXT / PK / FK	Fecha de la observación en Colombia
shortwave_radiation_sum_mj_m2	REAL	Radiación de onda corta acumulada diaria
daylight_seconds	REAL	Duración diaria de luz
sunshine_seconds	REAL	Duración diaria de sol informada por la fuente
matched_solar	INTEGER	La combinación ciudad-fecha encontró correspondencia solar
radiation_kwh_m2_day	REAL	shortwave_radiation_sum_mj_m2 / 3.6
daylight_hours	REAL	daylight_seconds / 3600
sunshine_hours	REAL	sunshine_seconds / 3600

8.1 Un registro solar por ciudad y día

La clave primaria (city_id, local_date) permite guardar un solo registro solar por ciudad y día. Antes de separar estos datos, model.py comprueba que las 24 filas horarias de cada día tengan los mismos valores solares. Si encuentra diferencias, detiene el proceso; si coinciden, guarda una sola fila diaria.

Para sumar la radiación del año uso fact_solar_day y agrupo por city_id. En la vista enriched_data, el valor diario aparece en cada hora, así que sumarlo allí duplicaría la medida. La energía sintética sí está calculada por hora y se puede sumar por periodos, recordando que sigue siendo una simulación.

8.2 Integridad del dato diario

Las reglas CHECK exigen radiación igual o mayor que cero, luz diaria entre 0 y 86.400 segundos, y horas de sol que no superen la duración de la luz. Las dos claves foráneas comprueban que existan la ciudad y la fecha. Guardo también las conversiones a horas y kWh/m²/día para facilitar las consultas; sus fórmulas se revisan en las pruebas de enriquecimiento.

Los datos meteorológicos corresponden a 2025. La fecha UTC que aparece en los reportes indica cuándo se ejecutó el proceso. Son dos fechas distintas: una corresponde a la observación y la otra a la ejecución del código.

9. Tablas y campos: datos por hora

fact_hourly | 43.800 filas

Campo	Tipo / clave	Descripción
source_row	INTEGER / PK	Identificador de fila asignado por el proceso de EA2
city_id	INTEGER / FK	Identificador de ciudad en EA1
time_id	INTEGER / FK	Identificador de la hora en la dimensión temporal de EA1
temperature_c	REAL	Temperatura meteorológica horaria de EA1
humidity_pct	REAL	Humedad relativa horaria
precipitation_mm	REAL	Precipitación del intervalo horario
wind_speed_kmh	REAL	Velocidad del viento horaria
energy_kwh	REAL	Valor energético sintético generado en EA1
synthetic_energy	INTEGER	Identifica la energía sintética
imputed_fields	TEXT	Campos imputados por EA2, separados por
quality_status	TEXT	Estado de calidad de EA2
is_outlier	INTEGER	Alguna variable quedó marcada como atípica por EA2
outlier_fields	TEXT	Variables señaladas, separadas por
outlier_temperature_c	INTEGER	Marca IQR de temperatura
outlier_humidity_pct	INTEGER	Marca IQR de humedad
outlier_precipitation_mm	INTEGER	Marca IQR de precipitación
outlier_wind_speed_kmh	INTEGER	Marca IQR de viento
outlier_energy_kwh	INTEGER	Marca IQR de energía sintética
temperature_zscore	REAL	Escalado de temperatura heredado de EA2, calculado por ciudad con todo 2025
energy_zscore	REAL	Escalado de energía heredado de EA2, calculado por ciudad con todo 2025
humidity_fraction	REAL	humidity_pct / 100
precipitation_log1p	REAL	ln(1 + precipitation_mm) sobre el valor numérico en mm
degrees_above_reference_c	REAL	max(temperature_c - reference_temperature_c, 0)
degrees_below_reference_c	REAL	max(reference_temperature_c - temperature_c, 0)
enrichment_status	TEXT	COMPLETE cuando los seis cruces tienen correspondencia y sus campos están presentes

source_row identifica cada fila y es la clave primaria. La combinación (city_id, time_id) también debe ser única. Los índices por time_id y fecha solar apoyan las consultas por tiempo. La vista enriched_data reúne las seis tablas y conserva las 63 columnas en el orden del CSV, incluidas las marcas de calidad y de cruce. El esquema completo está en [src/sql/schema.sql](#).

10. Herramientas y simulación de nube

Herramienta	Para qué la usé	Límite y posible mejora
DuckDB 1.5.5	Permite consultar con SQL las bases de EA1 y la entrada limpia del enriquecimiento.	Se ejecuta en una sola máquina, dentro del proceso.
SQLite (sqlite3)	Guarda el modelo final, sus claves y reglas en un archivo fácil de compartir.	Tiene límites cuando muchos usuarios escriben al mismo tiempo.
Pandas 2.3.3 / NumPy 1.26.4	Permiten leer los formatos, limpiar, unir datos y repetir la simulación.	Carga en memoria. El volumen actual cabe en una máquina.
PySpark	Lo considero una opción para repartir el trabajo si aumenta el volumen. No se usa aquí.	Requiere configurar particiones y ejecutores, y medir si compensa el esfuerzo.
GitHub Actions	Ejecuta las pruebas y el proceso con Ubuntu/Python 3.12 y guarda los resultados.	La máquina de ejecución es temporal; los datos necesitan un respaldo.
Openpyxl / lxml	Lectura y exportación XLSX; lectura de tablas HTML. XML usa el parser etree.	Los CSV y manifiestos permiten revisar los datos y su origen.

Elegí SQLite para el modelo final porque permite trabajar con relaciones y restricciones en un archivo que se puede abrir sin instalar un servidor. Conservé DuckDB para usar las bases de las actividades anteriores. Así puedo seguir el proceso completo y revisar el resultado con SQL (DuckDB Foundation, s. f.; SQLite, s. f.-c).

La nube se simula con carpetas que separan los JSON originales, los datos limpios, los enriquecidos y la base final. GitHub guarda el código y las copias de los datos, mientras GitHub Actions ejecuta el proceso en una máquina temporal. El alcance es académico: no se desplegaron servicios como S3 ni un clúster distribuido.

Usé Pandas porque los datos actuales caben en la memoria de un computador. Si el volumen crece, primero revisaría el consumo de RAM, los tiempos y la posibilidad de procesar por partes. PySpark sería una opción para repartir el trabajo entre varias máquinas, pero habría que justificar esa complejidad con mediciones. En esta entrega se explica como alternativa y no se ejecuta (pandas development team, s. f.-b; Apache Software Foundation, s. f.).

11. Automatización y reproducción

El archivo que ejecuta GitHub Actions está en `.github/workflows/bigdata.yml`, en la raíz del repositorio. Deje una copia dentro de `sebastian_clavijo_correa` para cumplir la estructura de la actividad; el proceso comprueba que sean iguales. Se activa al subir cambios, abrir una solicitud de cambios o iniciarlo manualmente. Usa `contents: read`, versiones fijas de las dependencias y acciones identificadas por su SHA.

Orden	Operación	Evidencia o condición
1	Checkout e instalación	Python 3.12; requirements.txt; setup.py
2	Pruebas unitarias	44 casos; build/tests.log
3	Repetir ingesta y comparar con EA1	5 JSON íntegros; valores idénticos
4	Limpieza y contraste EA2	CSV nuevo idéntico byte a byte
5	Enriquecimiento	6 cruces; CSV idéntico al antecedente
6	Modelo SQLite	6 tablas; 63 columnas reconstruidas
7	Publicación del artefacto	Salidas nuevas, auditorías, SQL, PDF y diagramas

Cada ejecución genera sus resultados en `build/`, una carpeta excluida de Git. De esta forma se comprueban salidas recién creadas y se conservan las bases de entrada. Si una comparación o una regla falla, el proceso se detiene y no publica el paquete de resultados.

11.1 Comandos desde la carpeta académica

```
python -m venv .venv
```

```
python -m pip install -r requirements.txt
```

```
python -m pip install -e . --no-deps --no-build-isolation
```

```
python -m unittest discover -s tests -v
```

```
python src/pipeline.py --output-dir build
```

```
python src/enrichment.py
```

Antes de instalar, activa el entorno con `.venv\Scripts\Activate.ps1` en PowerShell o `source .venv/bin/activate` en Linux/macOS.

`pipeline.py` ejecuta todo el proceso; `enrichment.py` permite repetir solo el enriquecimiento desde `cleaned.duckdb`. El README explica cómo clonar el repositorio y dónde queda cada salida.

Actions guarda el paquete de resultados durante 30 días. Por eso también conservo las evidencias de la entrega en el repositorio y en el ZIP. El archivo `docs/ENTREGA.md` contiene el enlace y el resultado de la ejecución remota revisada (GitHub, s. f.).

12. Resultados, conclusiones y mejoras

Comprobación local	Resultado observado
Comparación con EA1	43.800 filas reproducidas con valores exactos
Comparación con EA2 y enriquecimiento	Ambos CSV completos coinciden byte a byte
Columnas conservadas	32 variables heredadas intactas; 31 incorporadas
SQLite	5 ciudades, 365 fechas, 24 horas, 8.760 instantes, 1.825 hechos solares y 43.800 horarios
Integridad	0 huérfanos; integrity_check=ok; 2.759.400 celdas reconstruidas
Pruebas	44 casos aprobados: 13 de limpieza, 25 de enriquecimiento y 6 del modelo

Las pruebas del modelo usan casos con un dato solar contradictorio, un booleano inválido, una hora duplicada y una referencia a una fecha inexistente. El proceso debe rechazarlos. También revisan los ceros iniciales de los códigos DANE y la recuperación de todas las columnas. Las pruebas de las fases anteriores comprueban nulos, duplicados, unidades, cruces y zonas horarias. No se hicieron pruebas de carga distribuida ni de predicción.

12.1 Conclusiones

Con esta integración puedo seguir un dato desde la respuesta de la API hasta la base final. Las comparaciones con las actividades anteriores permiten comprobar que la limpieza y el enriquecimiento se mantienen al ejecutar todo junto. SQLite agrega reglas sobre las claves y separa los valores diarios de los horarios, lo que ayuda a consultar la información sin contar medidas de más.

El proyecto se limita a cinco ciudades, un año y una variable de energía simulada. Por eso no permite estimar la demanda eléctrica real de Colombia. Además, el proceso trabaja en memoria y guarda archivos locales: funciona para esta actividad, pero necesitaría cambios para manejar muchos más datos o varios usuarios al mismo tiempo.

12.2 Recomendaciones

Como siguiente paso, llevaría los archivos a un almacenamiento de objetos con versiones y guardaría los datos en Parquet, separados por fecha. También revisaría una base analítica administrada, los permisos de acceso, el manejo de credenciales y las copias de respaldo. Antes de pasar a PySpark mediría tiempos y memoria. Para actualizar la información, agregaría una ingesta que traiga solo los datos nuevos, con reintentos y revisión del esquema.

Si el proyecto continúa hacia la predicción, necesitaría mediciones reales de energía o mantener claro que se trata de una simulación. Separaría entrenamiento y evaluación por tiempo, recalcularía las escalas con entrenamiento y usaría solo la información disponible al momento de predecir. Estas recomendaciones quedan como trabajo pendiente.

Bibliografía

Apache Software Foundation. (s. f.). *Cluster mode overview*.
<https://spark.apache.org/docs/latest/cluster-overview.html>

DuckDB Foundation. (s. f.). *Why DuckDB*.
https://duckdb.org/why_duckdb

GitHub. (s. f.). *Workflow artifacts*.
<https://docs.github.com/en/actions/concepts/workflows-and-actions/workflow-artifacts>

Open-Meteo. (s. f.). *Historical weather API*.
<https://open-meteo.com/en/docs/historical-weather-api>

pandas development team. (s. f.-a). *pandas.DataFrame.merge*.
<https://pandas.pydata.org/pandas-docs/version/2.3/reference/api/pandas.DataFrame.merge.html>

pandas development team. (s. f.-b). *Scaling to large datasets*.
https://pandas.pydata.org/pandas-docs/version/2.3/user_guide/scale.html

SQLite. (s. f.-a). *SQLite foreign key support*.
<https://www.sqlite.org/foreignkeys.html>

SQLite. (s. f.-b). *Datatypes in SQLite*.
<https://www.sqlite.org/datatype3.html>

SQLite. (s. f.-c). *Appropriate uses for SQLite*.
<https://www.sqlite.org/whentouse.html>

Clavijo Correa, J. S. (2026a). *Preprocesamiento y limpieza de datos en plataforma de Big Data en la nube* [Código fuente y evidencias]. GitHub.
https://github.com/JSebastianCCorrea/Preprocesamiento_Limpieza_de_Datos_en_Plataforma_de_Big_Data_en_la_Nube

Clavijo Correa, J. S. (2026b). *Enriquecimiento de datos en plataforma Big Data en la nube* [Código fuente y evidencias]. GitHub.
https://github.com/JSebastianCCorrea/Enriquecimiento_Datos_Plataforma_Big_Data_Nube

Materiales y fuentes de datos

Para esta documentación revisé el código, las bases y las evidencias de las actividades anteriores. Los archivos FUENTES.md y src/sources/source_manifest.json registran las consultas al DANE, Nager.Date y Open-Meteo, las condiciones de uso y los cambios de formato. También se conservan las doce respuestas originales del enriquecimiento para repetir y revisar los cruces.